

SECURITY REVIEW

FOR MYSTIC FINANCE



mystic[★]

INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	SCOPE	04
03	AUDITORS	05
04	METHODOLOGY	06
05	RISK CLASSIFICATION	07
06	FINDINGS SUMMARY	08
07	DETAILED FINDINGS	09
	M-01 Temporary DoS of User Withdrawals Due to Partial Plume Withdrawals Creating Artificial Shortfall	09
	L-01 Inefficient Max-Percentage Check Prevents Using Valid Partial Capacity	11
	L-02 Restake() May Re-Stake Funds Already Reserved for User Withdrawals	14
	I-01 Wrong Amount Check in unstakeGov Uses Total Validator Stake Instead of This Contract's Stake	15
	I-02 ERC20 Transfer Handling in recoverERC20	16
08	DISCLAIMER	17
09	ABOUT KANN AUDITS	18

01 - OVERVIEW

AUDIT SUMMARY

Mystic Finance engaged Kann Audits to perform a security review of the Mystic Finance smart contracts. A team of 3 security researchers performed a manual review of the in-scope contracts. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

5

ISSUES

CRITICAL	-
HIGH	-
MEDIUM	01
LOW	02
INFORMATIONAL	02

ENGAGEMENT DETAILS

PREPARED FOR	Mystic Finance
PREPARED BY	Kann Audits
DATE REPORTED	9 December 2025
METHODS	Manual review

02 - WHAT WE REVIEWED

SCOPE

REPOSITORY	https://github.com/mystic-finance/Liquid-Staking
AUDITED COMMIT	e7af6a3201546962eedd17229181138ac9a5830a

#	FILE
01	stPlumeMinter.sol
02	OperatorRegistry.sol
03	stPlumeRewards.sol
04	frxETHMinter.sol
05	frxETH.sol
06	Periphery/MyPlumeFeed.sol
07	Utils/OwnedUpgradeable.sol

03 - THE TEAM ON THIS ENGAGEMENT

AUDITORS

KA

Kann

LEAD SECURITY RESEARCHER

JU

Jun Wang

LEAD SECURITY RESEARCHER

PI

Pindarev

INTERN

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01

Kickoff & Context

A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02

Scope & Preparation

Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03

AI First Scan

Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04

Expert Security Review

Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05

Findings & Proofs

Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06

Mitigation Review

We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- ✓ Testing against common and uncommon attack vectors
- ✓ Contract logic matching the client's intended behaviour
- ✓ Manual line-by-line review by experienced auditors
- ✓ Architecture and design analysis for secure component interactions and clear trust boundaries
- ✓ Compliance with industry best practices and known smart contract security standards
- ✓ Cross-referencing patterns with industry-standard implementations
- ✓ Access control verification
- ✓ State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

● CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
● HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
● MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
● LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
● INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

06 - ALL ISSUES

FINDINGS SUMMARY

● CRITICAL	● HIGH	● MEDIUM	● LOW	● INFO
-	-	01	02	02
TOTAL 05 FIXED 01 ACKNOWLEDGED 04				

ID	TITLE	SEVERITY	STATUS
M-01	Temporary DoS of User Withdrawals Due to Partial Plume Withdrawals Creating Artificial Shortfall	● MEDIUM	● FIXED
L-01	Inefficient Max-Percentage Check Prevents Using Valid Partial Capacity	● LOW	● ACKNOWLEDGED
L-02	Restake() May Re-Stake Funds Already Reserved for User Withdrawals	● LOW	● ACKNOWLEDGED
I-01	Wrong Amount Check in unstakeGov Uses Total Validator Stake Instead of This Contract's Stake	● INFORMATIONAL	● ACKNOWLEDGED
I-02	ERC20 Transfer Handling in recoverERC20	● INFORMATIONAL	● ACKNOWLEDGED

DETAILED FINDINGS

M-01

Temporary DoS of User Withdrawals Due to Partial Plume Withdrawals Creating Artificial Shortfall

SEVERITY

● MEDIUM

STATUS

● FIXED

DESCRIPTION

When multiple users interact with the unstake and withdraw logic at different times, a user can be temporarily DoS'd from withdrawing their own funds because of how `plumeStaking.withdraw()` and the local deficit check are combined.

Consider this flow:

User A calls unstake with 50 ETH. This amount gets unstaked from Plume once either `withdrawalQueueThreshold` or `nextBatchUnstakeTimePerValidator` is reached.

User B calls unstake with a larger amount, for example 350 ETH. It takes at least four validators to satisfy this (assuming each has around 100 ETH capacity). Eventually, the full 350 ETH for User B is also unstaked on Plume. At this point, Plume holds around 400 ETH of unstaked funds belonging to both User A and User B.

User A then calls withdraw. Inside `_withdraw`, the contract executes:

```
if (totalWithdrawable > 0) {
    uint256 balanceBefore = address(this).balance;
    plumeStaking.withdraw();
    uint256 balanceAfter = address(this).balance;
    withdrawn = balanceAfter - balanceBefore;
}
```

This `plumeStaking.withdraw()` call pulls all 400 ETH of cooled funds from Plume into the contract's balance. Plume now shows zero withdrawable ETH, but the contract's internal buffers reflect that ETH.

Later, other users (call them User Group M) also unstake their funds and get cooling/parked. Once their cooldowns finish, their amounts become withdrawable on Plume too.

Now User B calls withdraw. The code sees `totalWithdrawable > 0` again, so it calls `plumeStaking.withdraw()` a second time. This time, only the newly cooled funds from User Group M are withdrawn from Plume and assigned to withdrawn. This withdrawn value is less than the 350 ETH that User B expects.

The function then hits the deficit logic:

```
if (withdrawn > 0 && withdrawn < amount) {  
    require(amount - withdrawn < fee, "Insufficient funds to cover deficit");  
}
```

Here, amount is User B's requested 350 ETH, and withdrawn is the much smaller amount that just came from User Group M. The condition `amount - withdrawn < fee` will almost always be false because the fee is tiny compared to the missing difference. As a result, the withdrawal reverts.

At this point, User B is effectively DOS'ed from withdrawing. Every time they try, they will again pull only the small cooled balance from Plume, fail the same `require(amount - withdrawn < fee)` check, and revert. They remain stuck until either another user with a withdrawal request calls `withdraw` first and drains the Plume balance in a way that avoids the partial-withdraw branch, or enough new unstake events accumulate so that the next `plumeStaking.withdraw()` call returns at least 350 ETH in one shot.

So even though User B has correctly unstaked and the system as a whole has enough ETH across buffers `(currentWithheldETH, totalInstantUnstaked)` and future withdrawals, the specific interaction between `plumeStaking.withdraw()` and the `require(amount - withdrawn < fee)` deficit check can temporarily lock that user out of claiming their funds.

Additionally, User B would normally be allowed to withdraw using `currentWithheldETH` alone if the `totalWithdrawable` from plume was 0. However, because `withdrawn` is non-zero, the contract is forced into the deficit branch instead of using local liquidity because `withdrawn < amount`. This guarantees a revert as soon as enters check `withdrawn < amount`, making the DOS unavoidable until conditions change.

■ REMEDIATION ● FIXED

L-01

Inefficient Max-Percentage Check Prevents Using Valid Partial Capacity

SEVERITY

● LOW

STATUS

● ACKNOWLEDGED

DESCRIPTION

In the auto-distribution path of `_depositEther`, the contract decides whether a validator can be used by calling `getNextValidator(remainingAmount, validatorId)` with the full

```
remainingAmount:

while (remainingAmount > 0 && index < numVals) {
    uint256 depositSize = remainingAmount;
    _validatorId = uint16(validators[index].validatorId);
    (uint256 validatorId, uint256 capacity) = getNextValidator(remainingAmount,
        _validatorId);

    if (capacity > 0 && capacity ≥ minStakeAmount) {
        if (capacity < depositSize) {
            depositSize = capacity;
        }

        if (depositSize < minStakeAmount) {
            currentWithheldETH += remainingAmount;
            return depositedAmount;
        }

        plumeStaking.stake{value: depositSize}(uint16(validatorId));
        remainingAmount -= depositSize;
        depositedAmount += depositSize;
        emit DepositSent(uint16(validatorId));
    }
    index++;
}
```

Inside `getNextValidator`, the validator's new percentage is computed using `depositAmount` equal to that full `remainingAmount`:

```
function getNextValidator(uint256 depositAmount, uint16 validatorId)
    internal
    view
    returns (uint256 validatorId_, uint256 capacity_)
{
    (bool active, , uint256 stakedAmount, ) = plumeStaking.getValidatorStats(
        validatorId);
    uint256 totalStaked = plumeStaking.totalAmountStaked();
    if (!active) return (validatorId, 0);

    (, capacity_) = _getValidatorInfo(validatorId);

    uint256 percentage =
        ((stakedAmount + depositAmount) * RATIO_PRECISION) /
        (totalStaked + depositAmount);

    if (maxValidatorPercentage[validatorId] > 0 &&
        percentage > maxValidatorPercentage[validatorId])
    {
        return (validatorId, 0); // validator is skipped entirely
    }

    if (capacity_ > 0) {
        return (validatorId, capacity_);
    }
    return (validatorId, 0);
}
```

The critical line is:

```
if (maxValidatorPercentage[validatorId] > 0 && percentage > maxValidatorPercentage[
    validatorId]) {
    return (validatorId, 0);
}
```

If sending the entire `remainingAmount` would push a validator above its `maxValidatorPercentage`, `getNextValidator` returns a capacity of 0, and `_depositEther` skips that validator completely. However, `_depositEther` later caps `depositSize` to capacity only if capacity > 0, so this partial capping never happens for validators that are rejected at the percentage check.

This means that if a validator still has some allowed headroom under both capacity and max-percentage, but not enough to take the full `remainingAmount`, the contract refuses to use it at all. It does not attempt a partial deposit that would keep the validator within limits.

For example, a validator with max capacity 10 ETH and current stake 8 ETH still has 2 ETH of safe room. If `remainingAmount` is 5 ETH, the percentage is calculated as if 5 ETH were going to this validator. If that hypothetical 5 ETH would violate the max percentage, the validator is excluded, even though putting just 2 ETH there would be perfectly valid. As a result, those 2 ETH are never used for this validator; the loop just moves to the next one with the full 5 ETH still in `remainingAmount`.

Validators that still have valid headroom (for example, 1–2 ETH) can be skipped entirely if the full `remainingAmount` would break the percentage limit.

This leads to under-utilization of individual validator capacities and reduces how evenly stake is spread, since the allocator refuses to use small remaining slots on validators that are close to their limit.

In practice, more ETH may end up routed to fewer validators.

■ **REMEDIATION** ● **ACKNOWLEDGED**

L-02

Restake() May Re-Stake Funds Already Reserved for User Withdrawals

SEVERITY

● LOW

STATUS

● ACKNOWLEDGED

■ DESCRIPTION

The `restake()` function can reuse funds that users have already unstaked and are waiting to withdraw. When users call `unstake()`, their ETH enters cooling/parked state under the contract, but `restake()` does not check whether those funds are already committed to pending withdrawal requests. It simply calls:

```
plumeStaking.restake(validatorId, amount);
```

and consumes whatever cooling balance exists.

This allows the rebalancer to restake ETH that users are expecting to withdraw after their cooldown period. If this happens, users may reach the end of their cooldown and find that their withdrawal can-not be fulfilled because their cooled funds were re-staked before they could withdraw them. This can lead to temporary withdrawal delays or failed withdrawals until the protocol unstake cycle produces new cooled funds again.

■ REMEDIATION

● ACKNOWLEDGED

I-01

Wrong Amount Check in unstakeGov Uses Total Validator Stake Instead of This Contract's Stake

SEVERITY

● INFORMATIONAL

STATUS

● ACKNOWLEDGED

DESCRIPTION

The unstakeGov function currently does:

```
function unstakeGov(uint16 validatorId, uint256 amount)
    external
    nonReentrant
    onlyByOwnGov
    returns (uint256 amountRestaked)
{
    _rebalance();
    (bool active, , uint256 stakedAmount, ) =
        plumeStaking.getValidatorStats(uint16(validatorId)); // total stake on this
        validator

    if (active && stakedAmount > 0 && stakedAmount ≥ amount) {
        amountRestaked = plumeStaking.unstake(uint16(validatorId), amount);
    }
}
```

Here, `stakedAmount` is the total amount staked on the validator by all users, not the amount staked by this contract. The function should instead compare amount against this contract's own stake on that validator, using:

```
uint256 validatorStakedAmount =
    plumeStaking.getUserValidatorStake(address(this), validatorId);
```

so the precondition matches what this contract is actually allowed to unstake.

In practice, if amount is larger than this contract's stake, `plumeStaking.unstake` will revert internally, so funds are not at risk. However, the local check is misleading: it may pass when this contract's own stake is smaller, and the revert will only happen deeper in the staking contract. This makes the API less clear and can cause unnecessary reverts instead of failing fast with an accurate condition.

The suggested fix is to keep active from `getValidatorStats`, but replace the `stakedAmount` comparison with a comparison against `getUserValidatorStake(address(this), validatorId)`.

REMEDiation

● ACKNOWLEDGED

I-02

ERC20 Transfer Handling in recoverERC20

SEVERITY

● INFORMATIONAL

STATUS

● ACKNOWLEDGED

■ DESCRIPTION

The function uses:

```
require(IERC20(tokenAddress).transfer(to, tokenAmount), "recoverERC20: Transfer failed");
```

This assumes every ERC20 token returns a bool. Many tokens do not return a boolean at all, which will cause this call to revert even when the transfer would succeed.

Because of this, the contract may be unable to recover certain ERC20 tokens sent to it by mistake.

Recommendation

Use `SafeERC20.safeTransfer` instead. Safe transfer helpers correctly handle tokens that do not return a boolean.

■ REMEDIATION

● ACKNOWLEDGED

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.



EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.



AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.



FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.



INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.



KANN AUDITS
WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits